这个报错叫 **ConcurrentModificationException（并发修改异常）**

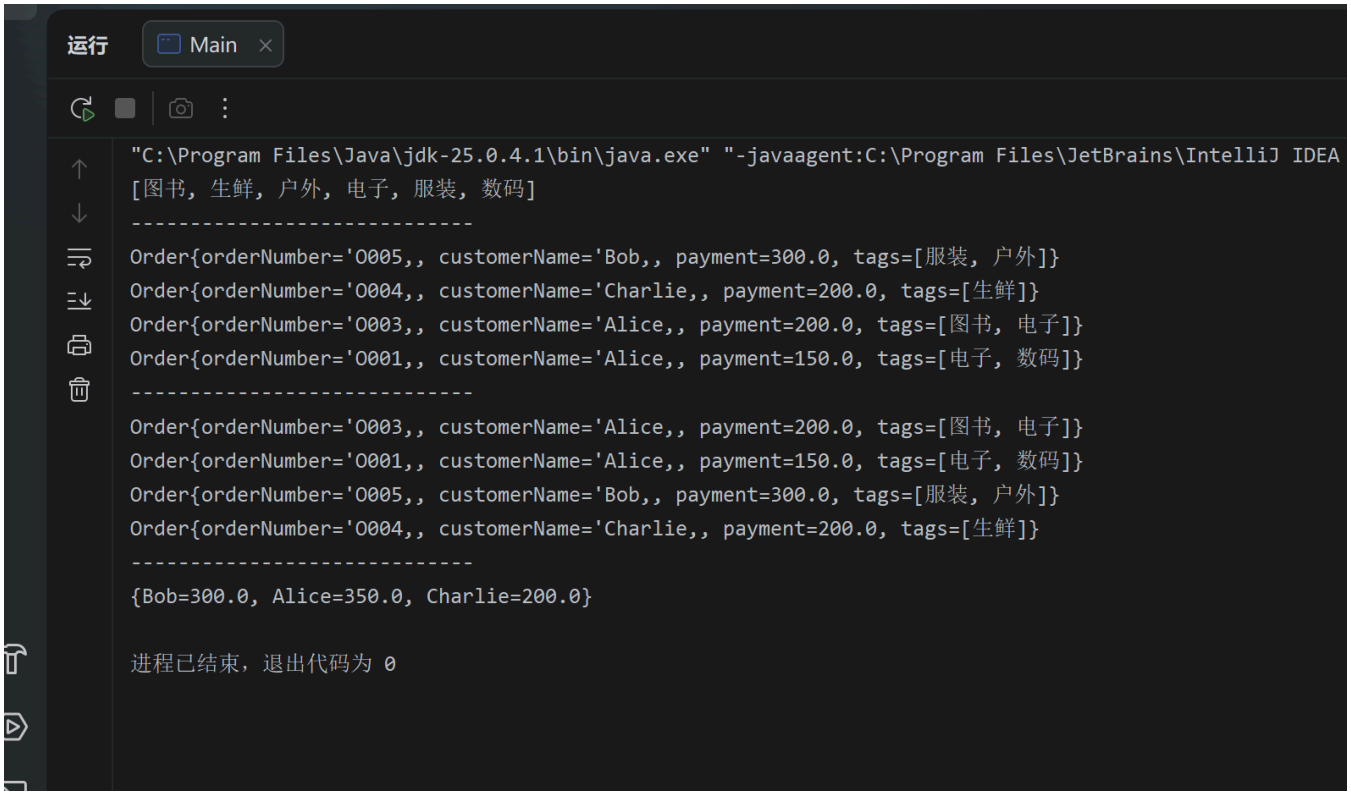
原因一句话：

增强 for 循环（for-each）遍历 ArrayList 的时候，不能直接用 `list.remove()` 删除元素！

底层迭代器在遍历，你直接删，它就检测到集合被偷偷改了，直接抛异常。

```java
// ❌ 错误写法（你现在的代码）
for(Order order : orders){
    if(order.getPayment() <=0){
        orders.remove(order);  // 遍历的时候 remove，炸！
    }
}
```

运行结果：




```java
package Bao6;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        //创建订单分析器对象
        OrderAnalyzer analyzer = new OrderAnalyzer();
        analyzer.add(new Order("Alice","O001",150.0, List.of("电子","数码")));
        analyzer.add(new Order("Bob","O002",-50.0, List.of("服装")));
        analyzer.add(new Order("Alice","O003",200.0, List.of("图书","电子")));
        analyzer.add(new Order("Charlie","O004",200.0, List.of("生鲜")));
        analyzer.add(new Order("Bob","O005",300.0, List.of("服装","户外")));
        //删除金额小于等于0的订单
        analyzer.removeInvalidOrders();
        Set<String> set = analyzer.getAllUniqueTags();
```

```
        //打印不重复的标签
        System.out.println(set);
        System.out.println("------------------------------");
        //写null是调用里面对象的compareTo方法排序，就是按金额从高到低排序，一样就按照订单号
        analyzer.getOrders().sort(null);
        //打印排序后的订单
        analyzer.printOrders();
        System.out.println("------------------------------");
        //按照用户名字排序
        analyzer.sortOrderByCustomerName();
        //打印排序后的订单
        analyzer.printOrders();
        System.out.println("------------------------------");
        //获取每个客户的订单金额，一一对应
        Map<String,Double> map = analyzer.getCustomerSpendMap();
        //打印每个客户的订单金额，一一对应
        System.out.println(map);

    }
}
```

```
package Bao6;

import java.util.Comparator;
import java.util.List;

public class Order implements Comparable<Order> {
    private String orderNumber;
    private String customerName;
    private double payment;
    private List<String> tags;
    //重写toString方便打印
    @Override
    public String toString() {
        return "Order{"+
                "orderNumber='" + orderNumber + ',' +
                ", customerName='" + customerName + ',' +
                ", payment=" + payment +
                ", tags=" + tags +
                '}';
    }
    //默认按金额从高到低排序，一样就按照订单号
    @Override
    public int compareTo(Order o){
        //比较金额
        int compare= Double.compare(o.getPayment(), this.getPayment());
        if(compare!=0){
            return compare;
        }
        //比较订单号
        return o.getOrderNumber().compareTo(this.getOrderNumber());
    }
```

```
    //构造方法
    public Order(String customerName, String orderNumber, double payment, List<String> tags)
{
        this.customerName = customerName;
        this.orderNumber = orderNumber;
        this.payment = payment;
        this.tags = tags;
    }

    public String getCustomerName() {
        return customerName;
    }

    public void setCustomerName(String customerName) {

        this.customerName = customerName;
    }

    public String getOrderNumber() {
        return orderNumber;
    }

    public void setOrderNumber(String orderNumber) {
        this.orderNumber = orderNumber;
    }

    public double getPayment() {
        return payment;
    }

    public void setPayment(double payment) {
        this.payment = payment;
    }

    public List<String> getTags() {
        return tags;
    }

    public void setTags(List<String> tags) {
        this.tags = tags;
    }
}
```

```
package Bao6;

import java.util.*;

public class OrderAnalyzer {
    //创建 集合储存信息，并禁止外界访问
    private List<Order> orders=new ArrayList<>();
    //遍历集合
```

```
    public void printOrders(){
        for(Order order : orders){
            System.out.println(order);
        }
    }
    //按金额从高到低排序，一样就按照订单号
    public List<Order>   paymentSort(){

 orders.sort(Comparator.comparing(Order::getPayment).reversed().thenComparing(Order::getOrde
rNumber));
        return orders;
    }
    //通过方法访问集合
    public List<Order> getOrders() {
        return orders;
    }

    public void setOrders(List<Order> orders) {
        this.orders = orders;
    }
    //添加订单
    public void add(Order order){
        orders.add(order);
    }
    //删除金额小于等于0的订单
    public void removeInvalidOrders(){
    //先储存到另一个集合中最后删除原集合的
        List<Order> list=new ArrayList<>();
        for(Order order : orders){
            if(order.getPayment()<=0){
                list.add(order);
            }
        }
        orders.removeAll(list);
    }
    //获取不重复的标签
    public Set<String> getAllUniqueTags(){
        Set<String> set=new HashSet<>();
        for(Order order : orders){
            set.addAll(order.getTags());
        }
        return set;
    }
    //获取每个客户的订单金额，一一对应
    public Map<String,Double> getCustomerSpendMap(){
        Map<String, Double> map = new HashMap<>();
        for(Order order : orders){
     //判断是不是已经有了，如果有了就直接加，没有就新加上
            if(map.containsKey(order.getCustomerName())){
                map.put(order.getCustomerName(),
map.get(order.getCustomerName())+order.getPayment());
            }else{
                map.put(order.getCustomerName(), order.getPayment());
```

```
            }
        }
        return map;
    }
    //按照用户名字排序
    public List<Order> sortOrderByCustomerName(){
        orders.sort(Comparator.comparing(Order::getCustomerName));
        return orders;
    }
}
```